# Practical - 12

## Practical Title: Event Management CRUD with Poster Upload and File Validation

Problem Definition: Develop event management functionality for admin users. Admin can add, view, update, delete, and upload event posters with file type and file size validation.

## Theory

**1. Are add, update, and delete functionalities implemented correctly?**

**Answer:** Yes. The CRUD (Create, Read, Update, Delete) functionalities are implemented to manage event records. The admin can add new events, view existing events, update event details, and delete events when required. These operations are performed using PHP and MySQL, with SQL queries used to insert, retrieve, update, and delete records from the events table. Appropriate success and error messages help the admin understand whether an operation was completed successfully.

**Reference Link:**
**https://www.w3schools.com/php/php_mysql_insert.asp**
**https://www.w3schools.com/php/php_mysql_select.asp**
**https://www.w3schools.com/php/php_mysql_update.asp**
**https://www.w3schools.com/php/php_mysql_delete.asp**

**2. Is the UI linked with the database correctly?**
**Answer:** Yes. The admin interface is connected to the MySQL database through PHP. When an admin submits an event form, PHP processes the submitted data and stores it in the database. Existing event records are retrieved from MySQL and displayed on the interface. When an event is updated or deleted, the corresponding database record is modified or removed. This ensures that changes made through the user interface are reflected in the database.

**Reference Link:**
**https://www.w3schools.com/php/php_mysql_connect.asp**
**https://www.w3schools.com/php/php_mysql_intro.asp**
**https://www.w3schools.com/php/php_forms.asp**

**3. Are poster uploads validated for type and size?**

**Answer:** Poster uploads are validated on the server side before the files are saved. The application checks whether a file has been selected, whether its file type is permitted (for example, JPEG, PNG, or WebP), and whether its size is within the specified limit. The application should validate the actual file content and MIME type rather than relying only on the file extension. Invalid files are rejected, and an appropriate error message is displayed. Valid files are stored in the designated upload directory, and their paths can be saved in the database.
For secure implementation, the upload directory should be protected, filenames should be generated safely, and only authorized administrators should be allowed to upload posters.

**Reference Link:**
https://www.w3schools.com/php/php_file_upload.asp

**4. Are success and failure messages displayed?**

**Answer:** Yes. The application should display appropriate success and failure messages after each operation. For example, messages such as “Event added successfully,” “Event updated successfully,” and “Event deleted successfully” confirm completed operations. Error messages can inform the admin about missing fields, invalid poster formats, oversized files, database connection problems, or unsuccessful operations. These messages improve usability and help the admin identify and correct errors.

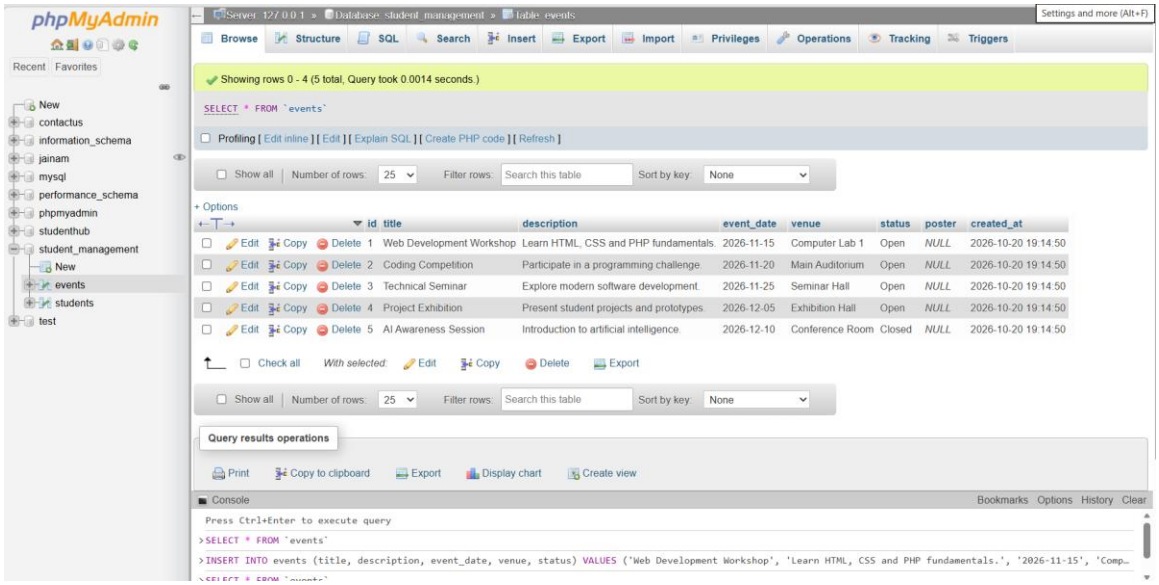
**Reference Link:**
https://www.w3schools.com/php/php_forms.asp
https://www.w3schools.com/php/php_echo_print.asp
https://www.w3schools.com/bootstrap5/bootstrap_alerts.php

## A. Implementation/ Output/Result

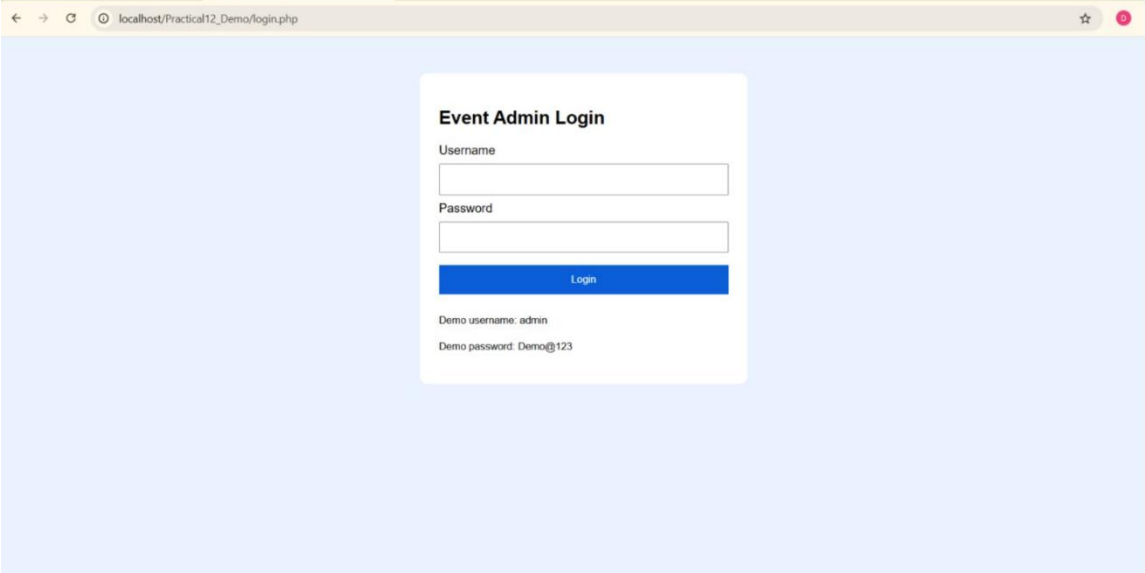
```
B.
C. <?php
D.
E. $host = "localhost";
F. $username = "root";
G. $password = "";
H. $database = "student_management";
I.
J. $conn = new mysqli(
K.     $host,
L.     $username,
M.     $password,
```

```
N.     $database
O. );
P.
Q. if ($conn->connect_error) {
R.     die("Connection failed: " . $conn->connect_error);
S. }
T.
U. ?>
V.
```

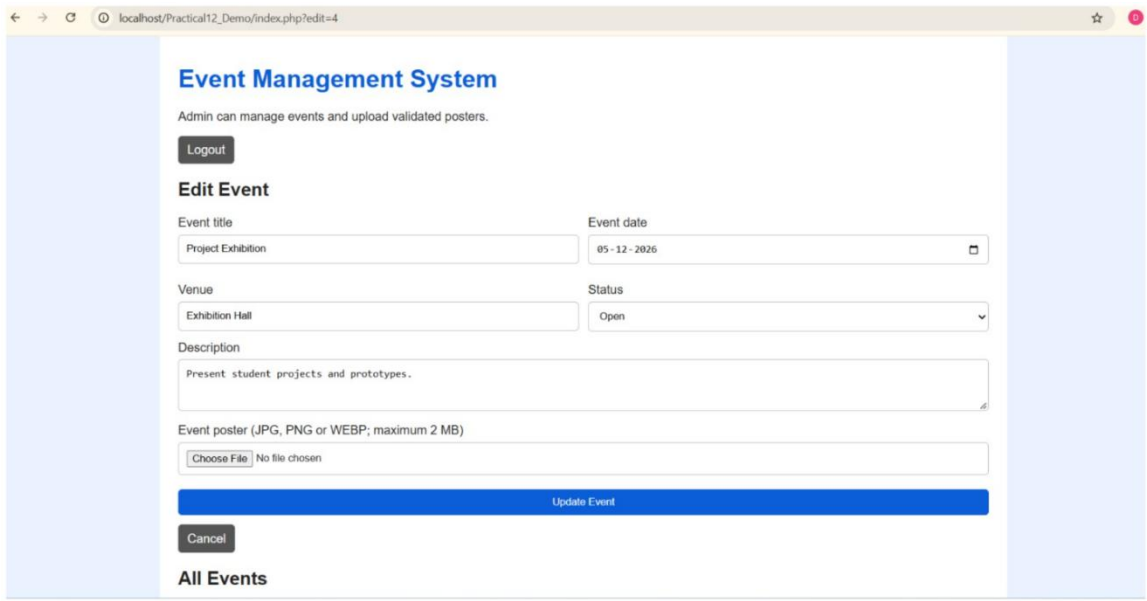
## B. Evidence of Work (Screenshots)




**Description:** This screenshot demonstrates the backend database of the Event Management System using phpMyAdmin. The events table stores event information such as ID, title, description, date, venue, status, poster, and creation timestamp. The records shown in the database correspond to the events displayed in the admin interface, confirming successful integration between the PHP application and MySQL database.

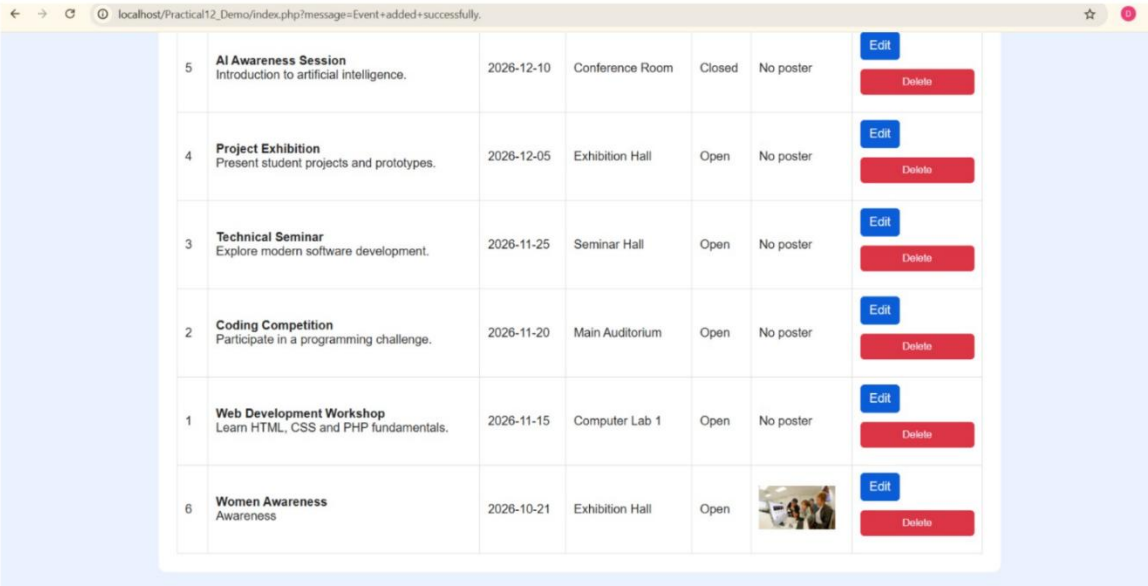



**Description:** The screenshot shows the Event Admin Login Page of the Event Management System developed using PHP and HTML/CSS. The page provides a secure login interface for administrators to access the event management dashboard. It contains username and password fields along with a Login button. Demo credentials are displayed for testing purposes. After entering valid credentials, the administrator can log in and manage event records. This login functionality helps restrict access to administrative features and ensures that only authorized users can perform CRUD operations, such as adding, viewing, updating, and deleting events.




**Description:** The screenshot shows the Edit Event section of the Event Management System. It allows the administrator to update existing event details stored in the MySQL

database. The form includes fields for the event title, event date, venue, status, and description. It also provides an option to upload an event poster in JPG, PNG, or WEBP format, with a maximum file size of 2 MB. The Update Event button saves the modified details to the database, while the Cancel button allows the administrator to cancel the editing process. The page also displays the All Events section, which supports event record management. This demonstrates the implementation of the Update operation in CRUD, database connectivity, event status management, and poster upload functionality with file validation.




**Description:** This screenshot demonstrates the admin-side Event Management interface. The system retrieves event records from the MySQL database and displays event information including title, description, date, venue, status, and poster. The Edit and Delete buttons provide update and delete functionality, while the successful event-added message demonstrates user feedback after a CRUD operation

## C. GitHub Repository Link:

https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical12_Demo

## D. Challenges Faced:

- Database connectivity: Establishing a proper connection between PHP and MySQL and resolving connection errors.
- CRUD operations: Ensuring that event records are correctly inserted, displayed, updated, and deleted.

- File upload validation: Restricting poster uploads to permitted file types and sizes while rejecting invalid files.

- File storage management: Saving uploaded posters in the correct directory and storing their paths in the database.

- Form validation: Preventing submission of incomplete forms and invalid event details.

- Error handling: Displaying clear success and failure messages for database and file upload operations.

- Access control: Ensuring that only authorized administrators can manage events and upload posters.

## E. Learning Outcomes:

- Implement CRUD operations using PHP and MySQL.

- Connect a web application to a MySQL database and manage event records.

- Upload poster images using PHP and validate file types and file sizes on the server side.

- Develop forms with input validation and appropriate error messages.

- Display event information dynamically using database records.

- Understand basic web application security, including access restrictions and safe file handling.

- Design a user-friendly admin interface using HTML, CSS, and Bootstrap or Tailwind CSS.

- Test database operations and file uploads to ensure that the application behaves correctly.

## Faculty Remarks: (Keep this blank for faculty)